

Veri Yapıları ve Algoritmalara Giriş

Bilgisayar mühendisliğinin temel taşlarından biri, veriyi organize etme (Veri Yapısı) ile o veri üzerinde işlem yapma (Algoritma) sanatını anlamaktır. Bu derste, performansın sadece "çalışıyor" olmadığını, aynı zamanda "ne kadar hızlı çalıştığı" ve "ne kadar bellek tükettiği" ile de ölçüldüğünü öğreneceksiniz.

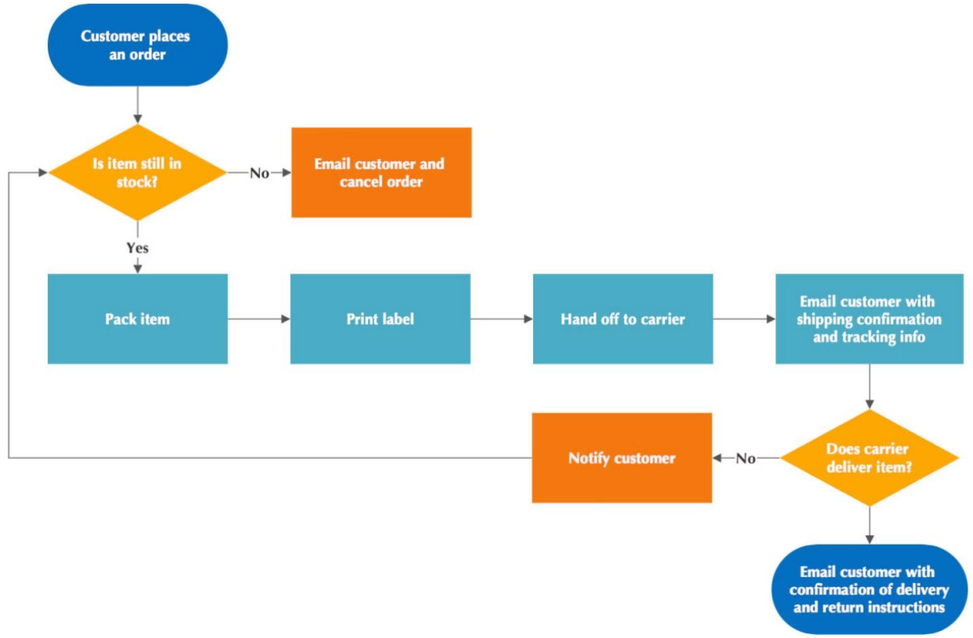
Modern yazılım geliştirmede başarılı olmak için, doğru veri yapısını seçmek ve verimli algoritmalar tasarlamak kritik öneme sahiptir. Bu bilgiler, kariyeriniz boyunca kullanacağınız temel yeteneklerdir.

Dersin Kapsamı ve Motivasyon

Problem Çözme Sanatı

Bilgisayar mühendisliği temelde problem çözmektir. Bir problemi çözmek için bir "yöntem" (algoritma) ve o yöntemin işleyeceği "malzeme" (veri) gerekir.

Algoritmalar **"ne kadar doğru, ne kadar hızlı, ne kadar az bellekle"** sorularına yanıt verir. Veri yapıları ise bu hedef için veriyi doğru şekilde temsil eder.



Bu Hafta Öğreneceksiniz:

- Dizi ve Bağlı Liste yapıları
- String işlemleri ve karakter kodlaması
- Arama ve Sıralama algoritmaları
- Performans analizi ve Big O notasyonu

"Ne Kadar Hızlı?" - Asimptotik Karmaşıklık

Bir programın 1.2 saniye çalışması donanıma bağlıdır. Ama "girdi boyutu (n) 2 katına çıkınca, süre 4 katına çıkıyor" demek donanımdan bağımsızdır ve algoritmanın gerçek karakterini gösterir.

O(1) - Sabit Zaman

Dizide indekse erişim. Girdi boyutu arttıkça süre değişmez.

O(n) - Doğrusal

Sıralanmamış dizide arama. Süre girdi boyutuyla doğru orantılı büyür.

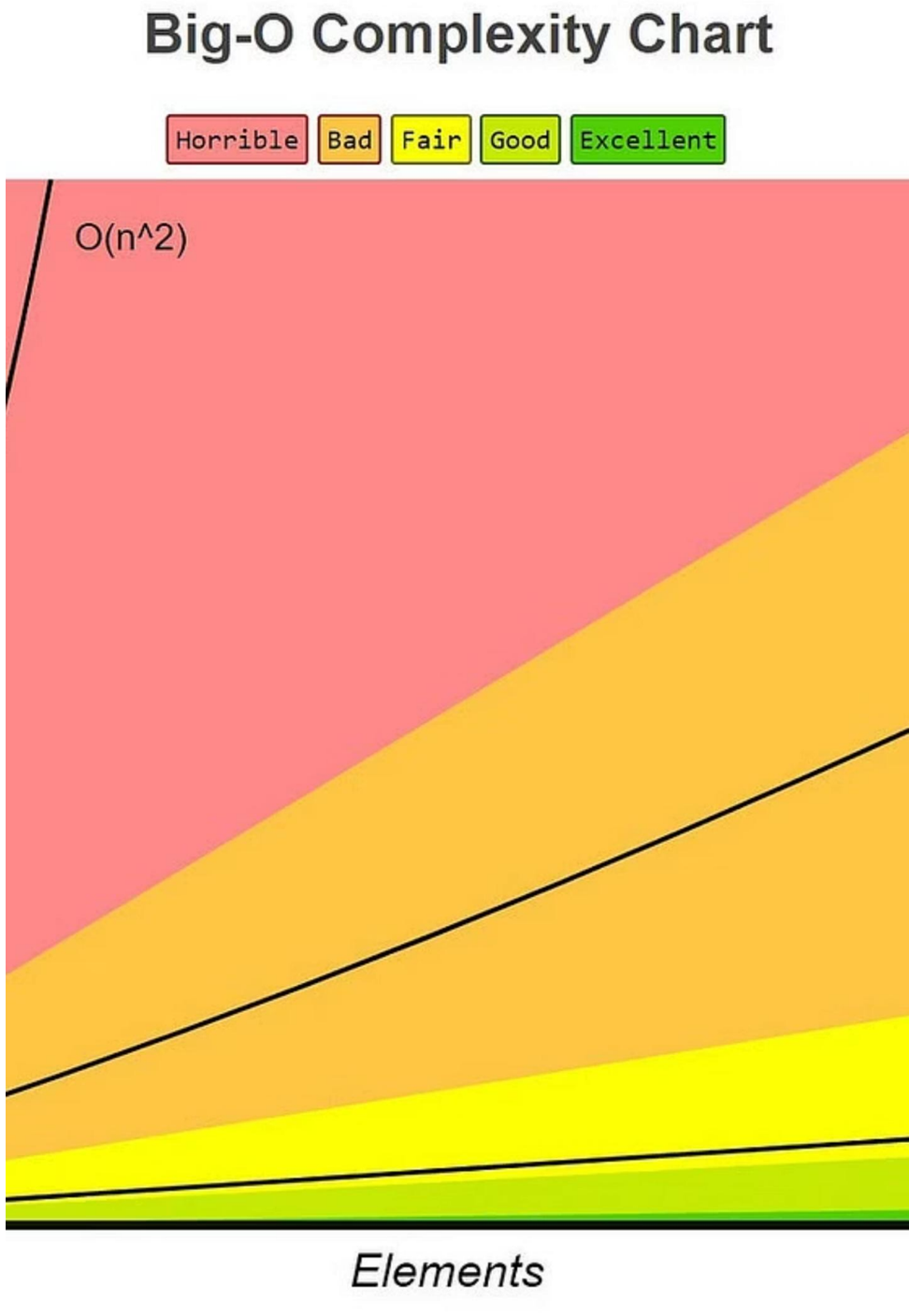
O(log n) - Logaritmik

İkili arama. Çok hızlı! Her adımda problem yarıya iner.

O(n²) - Karesel

İç içe döngüler. Girdi 2 katına çıkınca süre 4 katına çıkar.

Büyük O (Big O) notasyonu, algoritmanın performansının **"büyüme hızını"** anlatır. *O(n^2)*'nin *O(n)*'den her zaman daha yavaş büyüyeceğini biliriz. Bu bilgi, donanımdan bağımsız olarak algoritmaları karşılaştırmamızı sağlar.



Veri Yapısı 1: Dizi (Array)

Bilgisayar belleğindeki bitişik (arka arkaya dizilmiş) kutulardır. Her kutunun bir adresi (indeksi) vardır ve bu adres üzerinden anında erişim sağlanır.

Temel Özellikler

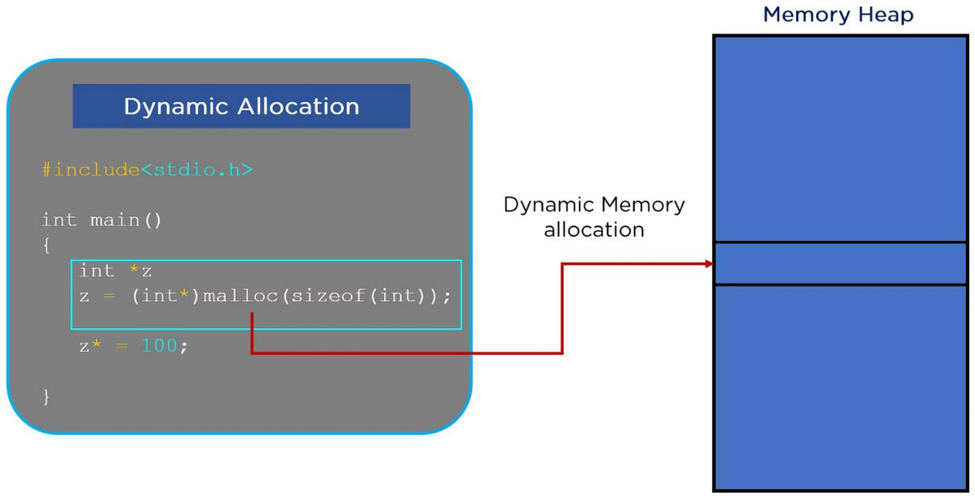
Sabit boyutlu, bitişik bellek yapısıdır. Bu fiziksel organizasyon, dizilere benzersiz performans özellikleri kazandırır.

Avantajları

- Rastgele erişim *O(1)*'dir
- dizi[100]'ün nerede olduğunu anında biliriz
- Önbellek (Cache) için harikadır
- Basit ve hızlı bellek yönetimi

Dezavantajları

- Araya eleman ekleme/silme pahalıdır: *O(n)*
- Geri kalan elemanları kaydırmak gerekir
- Boyut değiştirme esnekliği yoktur



Dinamik Dizi: Büyüyen Yapılar

Normal diziler "sabit boyutludur". Peki Python'daki `list.append()` veya C++'daki `vector` nasıl çalışır? Bu yapılar akıllıca bir strateji kullanır.

01

Başlangıç Durumu

Dizi belirli bir başlangıç kapasitesiyle oluşturulur (örneğin 4 eleman).

02

Kapasite Doluyor

Elemanlar eklenirken kapasite dolduğunda özel bir işlem başlar.

03

Yeni Alan Tahsisi

Genellikle 2 katı büyüklükte yeni bir bellek alanı istenir (8 eleman).

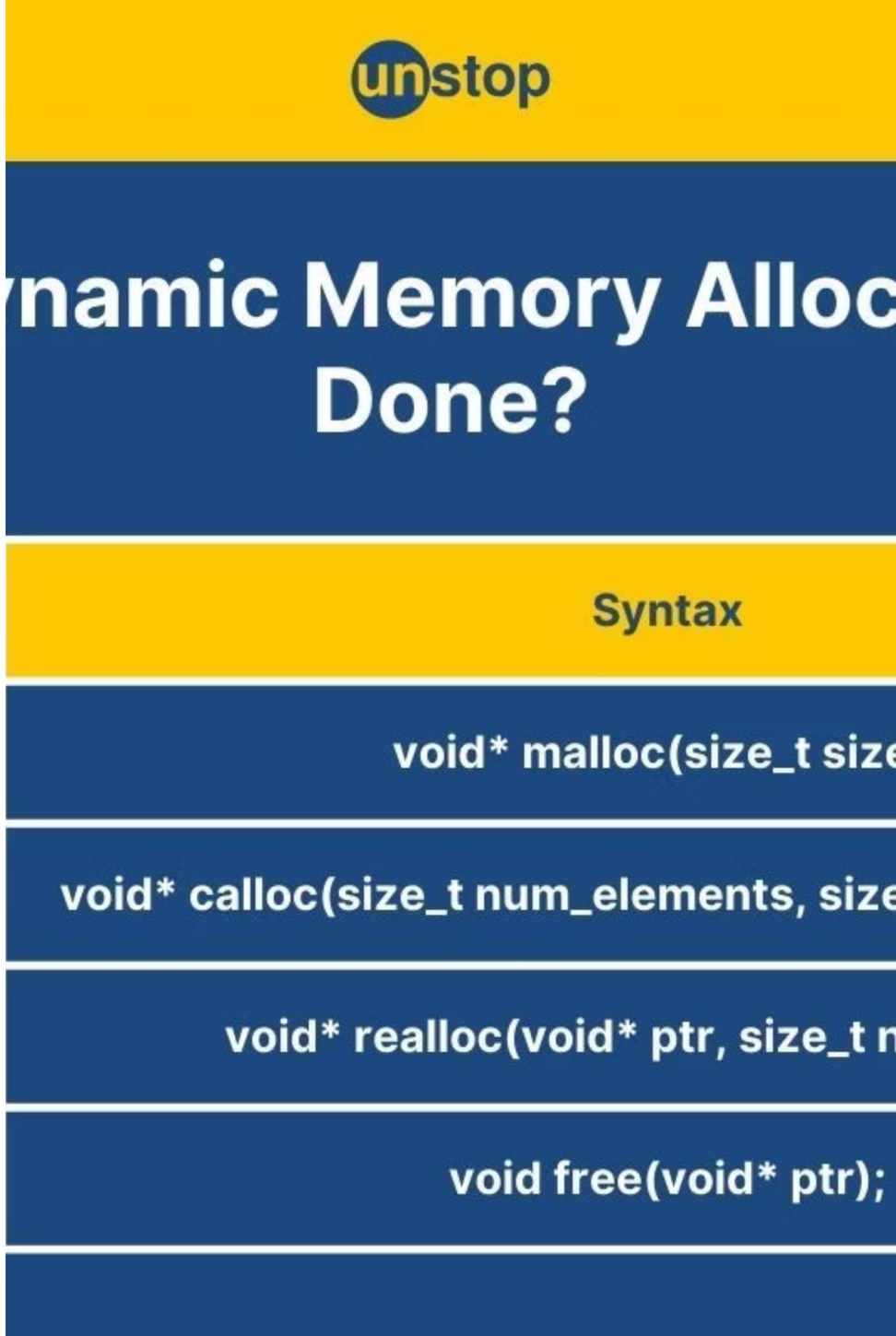
04

Kopyalama ve Silme

Tüm elemanlar yeni alana kopyalanır ve eski alan serbest bırakılır.

Amortize Analiz: Akıllı Maliyet Hesabı

Bu "kopyalama" anı pahalıdır (*O(n)* sürer), ancak çok nadiren olur. Çoğu ekleme işlemi sadece boş yere yazar. Bu sayede, eleman ekleme işleminin **ortalama maliyeti** *O(1)*'e, yani çok ucuza gelir. Buna "amortize sabit zaman" denir.



Veri Yapısı 2: Bağlı Liste (Linked List)

Bağlı liste, dizilerden tamamen farklı bir felsefe sunar: Elemanlar bellekte **dağınık** durur. Her eleman (düğüm), hem "veriyi" hem de "bir sonraki elemanın adresini" (işaretçi) tutar.

Yapı ve Mantık

Her düğüm iki bölümden oluşur: veri ve işaretçi (pointer). İşaretçi, bir sonraki düğümün bellek adresini gösterir.

Avantajları

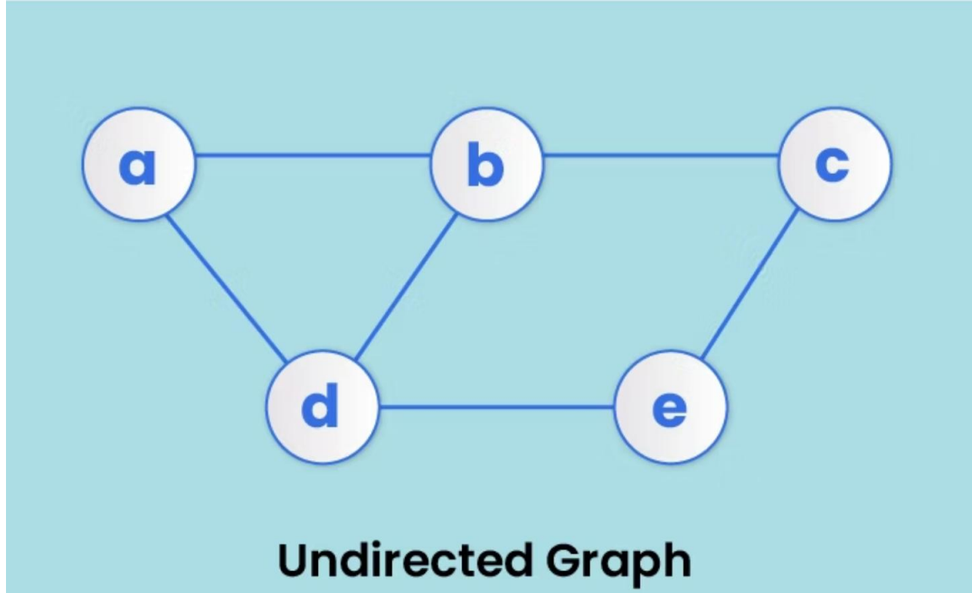
- Araya ekleme/silme *O(1)*'dir
- Sadece işaretçileri değiştirmek yeterli
- Kaydırma işlemi yoktur
- Dinamik boyut değişimi kolaydır

Performans Özellikleri

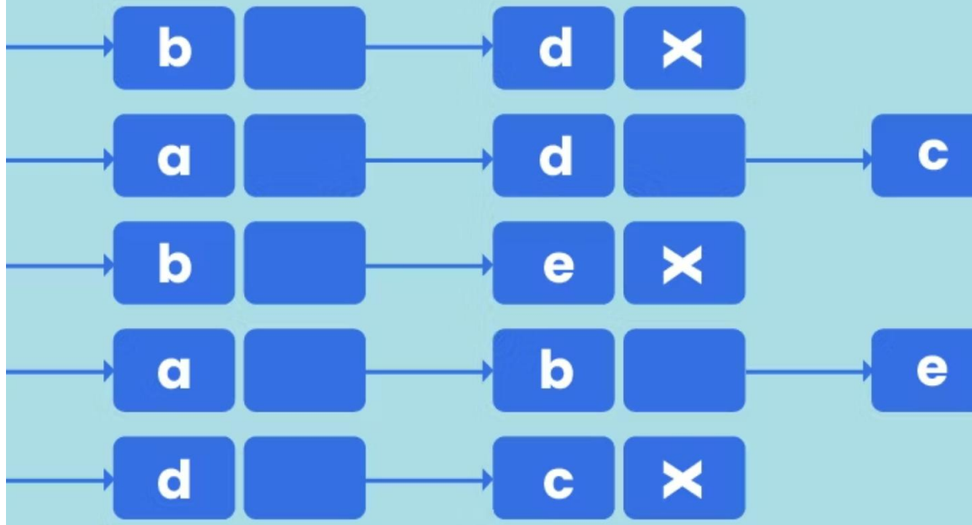
Dezavantajları

- Rastgele erişim *O(n)*'dir
- liste[100]'ü bulmak için baştan 100 adım gerekir
- Her düğüm ekstra bellek (işaretçi) tutar
- Önbellek (cache) için kötüdür

Bellek dağınık olduğu için, CPU'nun önbellek mekanizması bağlı listelerde etkisiz kalır.



Undirected Graph



Adjacency List

Doubly Linked List: Çift Yönlü Hareket

1

Tek Yönlü Liste

Sadece ileriye (next) gidebilirsiniz. Geriye dönmek için baştan başlamalısınız.

2

Çift Yönlü Liste

Hem ileriye (next) hem geriye (prev) hareket edebilirsiniz. Daha esnek ama daha fazla bellek kullanır.

Çift yönlü bağlı liste, tek yönlünün gelişmiş versiyonudur. Her düğüm, hem **"sonrakini" (next)** hem de **"öncekini" (prev)** bilir. Bu yapı, bazı işlemleri kolaylaştırır ancak düğüm başına bir işaretçi daha saklamak gerektiğinden bellek maliyeti artar.

Avantajlar

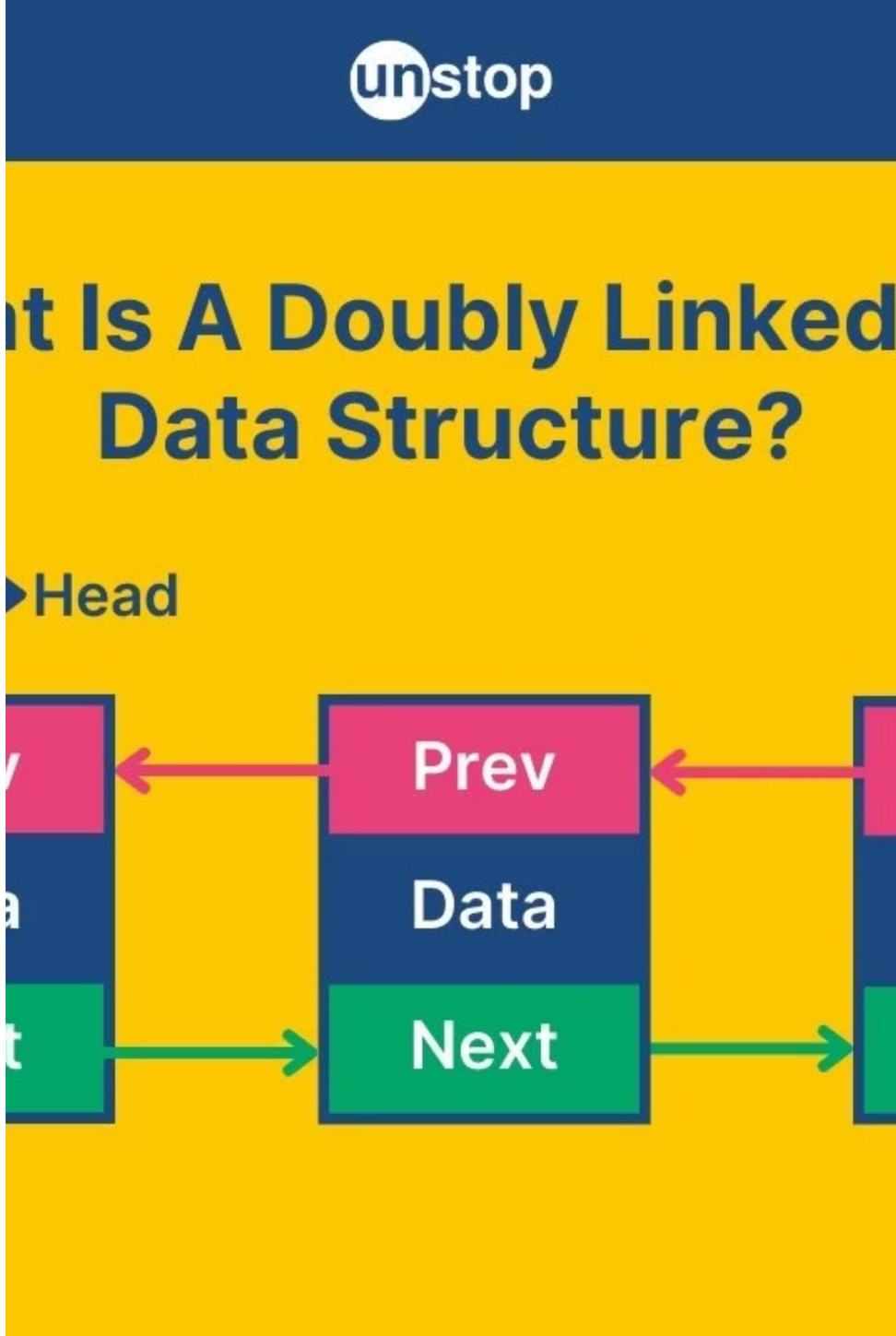
- Sondan silme *O(1)*
- İki yönlü gezinme
- Simetrik işlemler

Dezavantajlar

- Düğüm başına +1 işaretçi
- Daha karmaşık kod
- Bellek kullanımı artar

Kullanım Alanları

- Tarayıcı geçmişi
- Geri al/İleri al işlemleri
- Müzik çalar listeleri



Dizi vs. Bağlı Liste: Karşılaştırma

"Sihirli" bir veri yapısı yoktur. Her şey ihtiyaca ve trade-off'lara göredir. Doğru veri yapısını seçmek, yazılım geliştirmenin en kritik kararlarından biridir.

İşlem	Dizi	Bağlı Liste
Erişim (i. eleman)	O(1) ⚡	O(n)
Arama	O(n)	O(n)
Başa ekleme/silme	O(n)	O(1) ⚡
Sona ekleme	O(1) ⚡	O(n) veya O(1)*
Ortaya ekleme/silme	O(n)	O(1)* ⚡
Bellek verimliliği	Yüksek ⚡	Düşük (işaretçiler)
Cache performansı	Mükemmel ⚡	Zayıf

** Konum biliniyorsa veya tail pointer varsa*

Dizi Tercih Edin

Veriye çok hızlı erişmeniz gerekiyorsa (örn. dizi[i]) ve boyutu az çok biliyorsanız.

Bağlı Liste Tercih Edin

Sürekli araya eleman ekleyip çıkaracaksanız ve verinin sırası önemliyse.

Veri Yapısı 3: String (Karakter Dizisi)

String, temelde bir **"karakter dizisidir"**. Ancak göründüğünden çok daha karmaşıktır. "Merhaba" kelimesi bellekte nasıl saklanır? Bu basit soru, karakter kodlamasının önemini gösterir.

Karakter Kodlama

"A" harfi bellekte nasıl durur? Bu, kullanılan kodlamaya bağlıdır:

ASCII (7/8 bit)

- Sadece İngilizce karakterler
- Her karakter 1 byte
- 128 farklı karakter
- Türkçe karakterleri desteklemez

UTF-8 (Değişken uzunluk)

- Tüm dilleri destekler
- 1-4 byte arası değişken
- İngilizce: 1 byte
- Türkçe (ğ, ü, ş, ı, ö, ç): 2 byte

USASCII code chart

Bits b_7 b_6 b_5 →	0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
b_4 b_3 b_2 b_1 ↓ Row / Column →	0	1	2	3	4	5	6	7
0 0 0 0	0	NUL	DLE	SP	0	@	P	`	p
0 0 0 1	1	SOH	DC1	!	1	A	Q	a	q
0 0 1 0	2	STX	DC2	"	2	B	R	b	r
0 0 1 1	3	ETX	DC3	#	3	C	S	c	s
0 1 0 0	4	EOT	DC4	$	4	D	T	d	t
0 1 0 1	5	ENQ	NAK	%	5	E	U	e	u
0 1 1 0	6	ACK	SYN	&	6	F	V	f	v
0 1 1 1	7	BEL	ETB	'	7	G	W	g	w
1 0 0 0	8	BS	CAN	(	8	H	X	h	x
1 0 0 1	9	HT	EM	)	9	I	Y	i	y
1 0 1 0	10	LF	SUB	*	:	J	Z	j	z
1 0 1 1	11	VT	ESC	+	;	K	[	k	{
1 1 0 0	12	FF	FS	,	<	L	\	l	|
1 1 0 1	13	CR	GS	-	=	M	]	m	}
1 1 1 0	14	SO	RS	.	>	N	^	n	~
1 1 1 1	15	SI	US	/	?	O	_	o	DEL

Performans Farkındalığı

`string.length()` işlemi, string'in nasıl kodlandığına bağlı olarak *O(1)* veya *O(n)* sürebilir. UTF-8'de uzunluk bulmak için bazen tüm string'i taramak gerekir!

7

Merhaba (ASCII)

7 byte toplam bellek

8

Dünya (UTF-8)

6 byte (ü = 2 byte)

128

ASCII Karakter

Desteklenen toplam

1M+

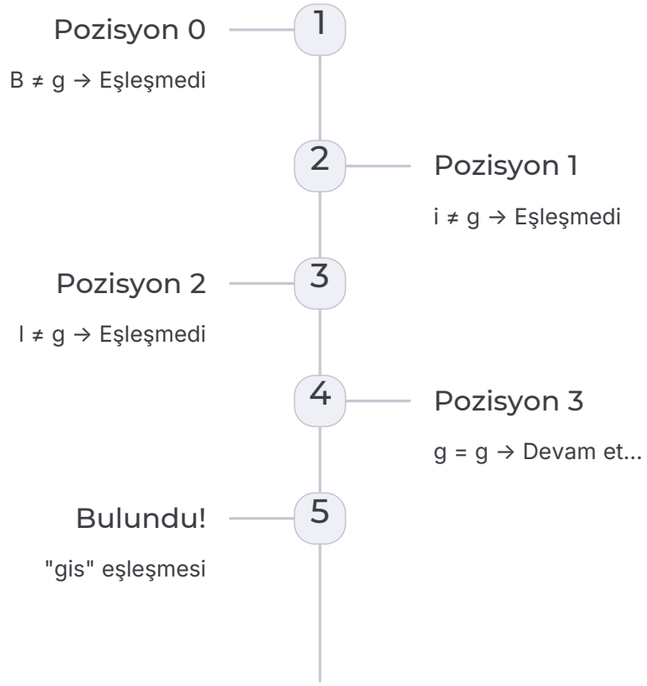
UTF-8 Karakter

Tüm diller desteklenir

String Algoritması: Alt Dize Arama

Naif Yaklaşım: Basit Ama Yavaş

"Bilgisayar" metni içinde "gis" kelimesini nasıl buluruz? En basit yöntem, her konumdan başlayarak eşleşme denemektir.



Algoritma Mantığı

1. Metnin her pozisyonundan başla
2. Aranan kelimenin her karakterini sırayla karşılaştır
3. Eşleşmezse bir sonraki pozisyona geç
4. Tam eşleşme bulunursa pozisyonu döndür

Performans: *O(n |cdot m)* - Metin uzunluğu × Aranan kelime uzunluğu. Basit ama büyük metinlerde yavaştır.

Örnek Karmaşıklık

1000 karakterlik metinde 10 karakterlik kelime aramak:

- En kötü durum: 10,000 karşılaştırma
- Ortalama: ~5,000 karşılaştırma

Gelişmiş Alt Dize Arama Algoritmaları

Daha Hızlısı Var!

Naif yaklaşım *O(n |cdot m)* karmaşıklığındaydı. Ancak algoritmik düşünce, çok daha verimli çözümler sunar. KMP, Rabin-Karp ve Boyer-Moore gibi algoritmalar, metinde "kayma" yaparken daha akıllıca davranarak *O(n+m)* gibi çok daha hızlı çalışabilir.

KMP (Knuth-Morris-Pratt)

Eşleşmeyen karakterde geriye dönmez. "Failure function" kullanarak akıllıca atlar. *O(n+m)* garantili.

Rabin-Karp

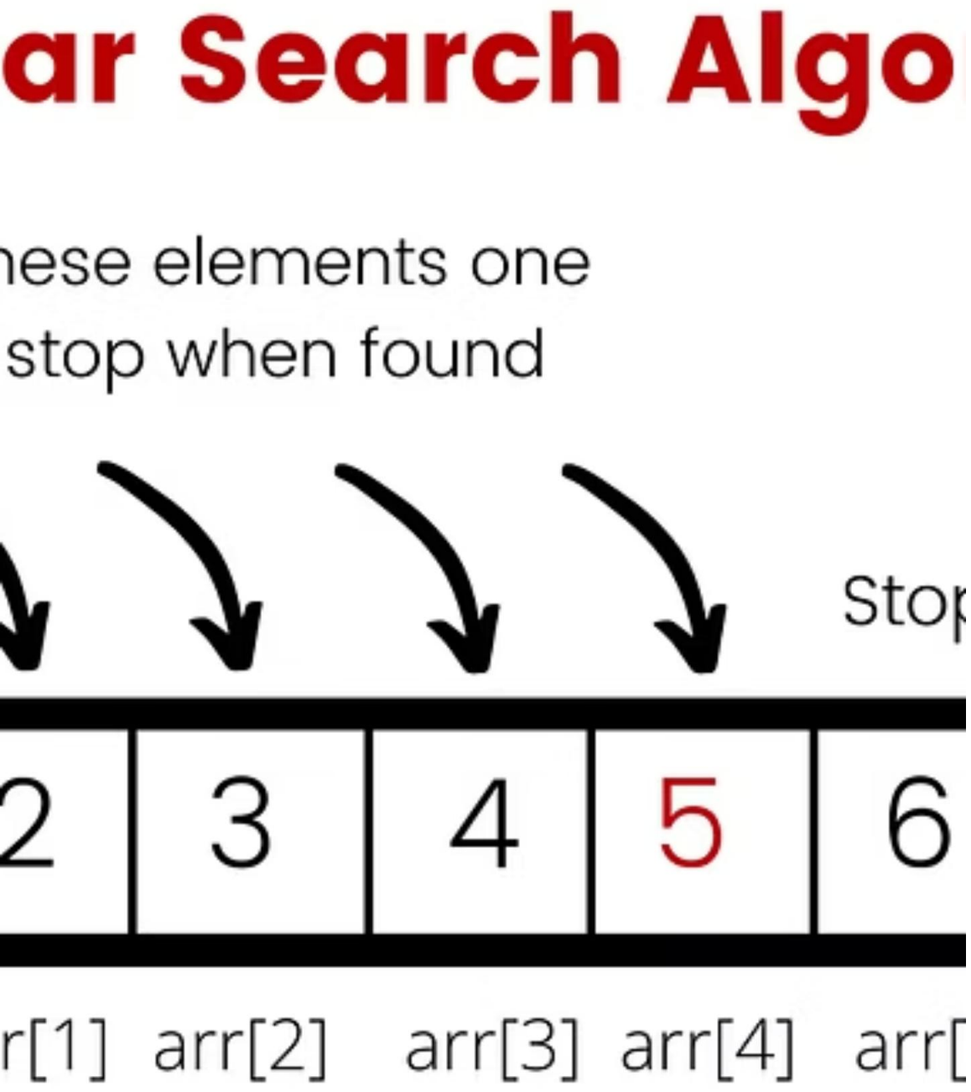
Hash fonksiyonları kullanır. "Rolling hash" ile hızlı karşılaştırma. Çoklu pattern aramada mükemmel.

Boyer-Moore

Sondan başa doğru karşılaştırır. "Bad character" ve "good suffix" kuralları. Pratikte en hızlısı.

Algoritmik Düşüncenin Gücü

Bu derste bu algoritmaların detaylarına girmeyeceğiz, ancak **"daha hızlısının mümkün olduğunu"** bilmek, algoritmik düşüncenin temelidir. Veri yapısı ve algoritma seçimi, performansı dramatik olarak değiştirir. Aynı probleme farklı yaklaşımlar, milisaniye ile saniye arasındaki fark yaratabilir.



Algoritma 1: Doğrusal Arama

Sıralanmamış bir dizide (veya listede) bir elemanı bulmanın **tek garantili yolu** doğrusal aramadır (Linear Search). Baştan sona, tek tek her elemanı kontrol ederiz.

Algoritma

Dizinin başından başla, her elemanı aranan değerle karşılaştır. Bulunursa indeksi döndür, bulunamazsa -1 döndür.

En İyi Durum

O(1) - İlk eleman aradığımız elemansa, tek karşılaştırmayla buluruz.

Ortalama Durum

O(n) - Ortalamada dizinin yarısını taramak gerekir. n/2 ≈ O(n).

En Kötü Durum

O(n) - Eleman dizinin sonundaysa veya hiç yoksa, tüm diziyi taramamız gerekir.

Avantajları

- Çok basit ve anlaşılır
- Sıralanmamış verilerle çalışır
- Küçük dizilerde yeterince hızlı
- Ekstra bellek gerektirmez

Dezavantajları

- Büyük dizilerde çok yavaş
- Verinin sıralı olmasından faydalanamaz
- Her aramada baştan başlar

Algoritma 2: İkili Arama (Binary Search)

Telefon Rehberi Mantığı

Telefon rehberinde bir ismi nasıl ararsınız? Sayfaları teker teker çevirmezsiniz. Ortadan açar, bakar ve hangi yarıda olduğuna karar verirsiniz.

Kritik Ön Koşul

Dizi **kesinlikle sıralı** olmalıdır! İkili arama, sıralı olmayan dizilerde çalışmaz.

01

Ortadaki Elemana Bak

Dizinin ortasındaki elemanı kontrol et.

02

Karşılaştır

Aradığın eleman ortadakinden büyükse sağ yarıya, küçükse sol yarıya git.

03

Aralığı Daralt

Seçilen yarıda aynı işlemi tekrarla. Her adımda arama alanı yarıya iner.

04

Bul veya Bitir

Eleman bulunana veya arama alanı bitene kadar devam et.

30

Maksimum Adım

1 milyar elemanlı dizide

20

Maksimum Adım

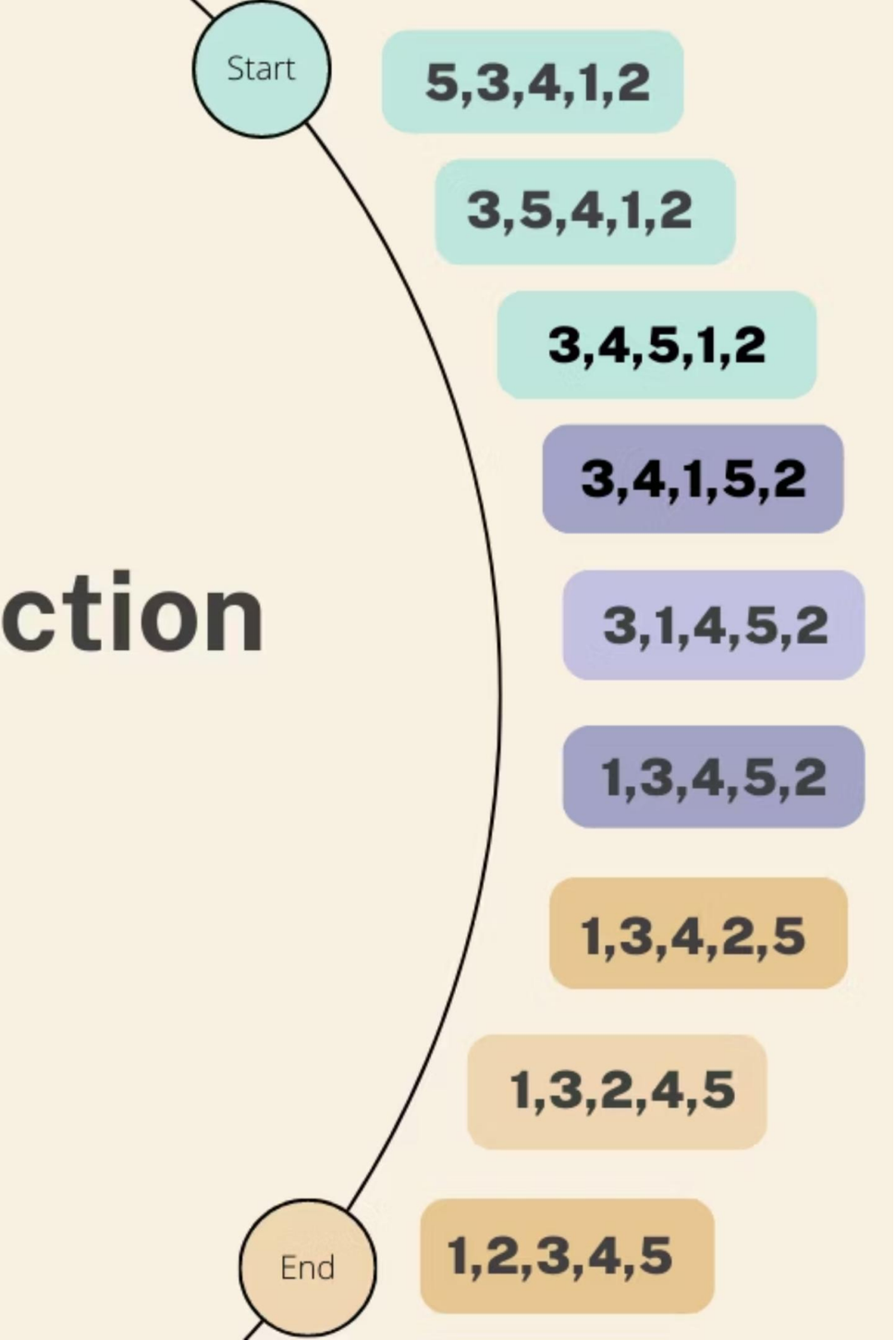
1 milyon elemanlı dizide

10

Maksimum Adım

1000 elemanlı dizide

Performans: *O(log n)* - Logaritmik karmaşıklık, büyük verilerde muazzam hız sağlar!



Seçmeli Sıralama (Selection Sort)

En basit sıralama mantıklarından biri. İnsanların "kartları sıralamasına" pek benzemez, ancak mantığı oldukça sezgiseldir.

Minimum Bul

Tüm listenin içinde en küçük elemanı bul.

Başa Taşı

Bulunan minimum elemanı listenin başındaki elemanla yer değiştir.

Kalan Liste

İlk elemanı "sıralı bölge" kabul et. Kalan listeye aynı işlemi uygula.

Tekrarla

Liste tamamen sıralanana kadar bu adımları tekrarla.

Karakteristik Özellikler

- Her zaman *O(n^2)* karmaşıklık
- Verinin sıralı olup olmaması önemli değil
- Yerinde (in-place) çalışır
- Stabil değildir
- n(n-1)/2 karşılaştırma yapar

Ne Zaman Kullanılır?

- Çok küçük dizilerde (n < 20)
- Bellek çok kısıtlıysa
- Kodun basitliği kritikse
- Performans önemli değilse

Gerçek Dünya: Pratikte nadiren kullanılır, çünkü aynı karmaşıklıkta daha iyi alternatifler var.

Eklemeli Sıralama (Insertion Sort)

Oyun Kartları Sıralama Yöntemi

Bu algoritma, elinizdeki oyun kartlarını sıralama şeklinize çok benzer. Doğal ve sezgisel bir yaklaşımdır.

1

Başlangıç

İlk eleman "sıralı bölge" kabul edilir.

2

Eleman Al

Sıralanmamış bölgeden bir eleman seçilir.

3

Doğru Yer Bul

Sıralı bölgede doğru pozisyon bulunur.

4

Yerleştir

Eleman doğru yerine kaydırılarak sokulur.

Performans Analizi

Ortalama/Kötü Durum: *O(n^2)* - İç içe döngüler nedeniyle karesel karmaşıklık.

En İyi Durum: *O(n)* - Neredeyse sıralı veride sadece tek geçiş yeterli! Bu özellik çok önemlidir.

Özel Durum: Adaptive (Uyarlanabilir)

Insertion Sort, verinin ne kadar sıralı olduğunu "algılar" ve buna göre hızlanır. Bu özelliğe "adaptive" denir.

Avantajları

- Basit ve anlaşılır
- Küçük dizilerde hızlı
- Yerinde çalışır
- Stabil sıralamadır
- Online algorithm
- Neredeyse sıralıda çok hızlı

Gerçek Dünya Kullanımı: Timsort (Python, Java) gibi hibrit algoritmalar, küçük parçalar için Insertion Sort kullanır çünkü bu boyutlarda çok verimlidir.

Merge Sort: Böl ve Fethet Stratejisi

"Böl ve Fethet" (Divide and Conquer) stratejisinin en temiz ve en zarif örneğidir. Algoritma tasarımında öğretilmesi gereken klasik bir yöntemdir.

1. Böl (Divide)

Diziyi tam ortadan ikiye böl. Sol ve sağ yarılar oluştur.

2. Fethet (Conquer)

Sol yarıyı rekürsif olarak sırala. Sağ yarıyı rekürsif olarak sırala.

3. Birleştir (Merge)

İki sıralı yarıyı akıllıca birleştirerek tek bir sıralı dizi elde et.

Temel Özellikler

- **Performans:** Her zaman *O(n |log n)*
- **Güvenilirlik:** En kötü durumda bile aynı hızda
- **Stabilite:** Stabil bir sıralama algoritmasıdır
- **Tahmin Edilebilirlik:** Girdi ne olursa olsun tutarlı

Merge İşlemi

İki sıralı diziyi birleştirmek *O(n)* sürer. İki işaretçi kullanarak, her adımda küçük olanı seçeriz.

Rekürsif Derinlik: *|log n* seviye vardır. Her seviyede *O(n)* işlem yapılır. Toplam: *O(n |log n)*

Dezavantajı: Bellek

Ekstra bellek *O(n)* kullanır. Birleştirme sırasında geçici diziler gerekir.

Kullanım Alanları

- Bağlı liste sıralama (ekstra bellek problemi yok)
- Dış sıralama (büyük dosyalar)
- Paralel sıralama
- Stabilitenin önemli olduğu durumlar

Quick Sort: Hızlı ve Popüler

Pratikte en çok kullanılan hızlı sıralama algoritmalarından biri. İsmi boşuna "Quick" değil - genelde gerçekten çok hızlıdır!

Pivot Seç

Diziden bir eleman seç (pivot/eksen). Genelde ortadaki veya rastgele bir eleman.



Partition

Pivottan küçükleri sola, büyükleri sağa ayır. Pivot doğru yerine gelmiş olur.

Rekürsif Çağrı

Sol ve sağ parça için aynı işlemi rekürsif olarak tekrarla.

Performans

Ortalama: *O(n |log n)*

En Kötü: *O(n^2)* (kötü pivot seçimi)

Bellek: *O(|log n)* veya *O(1)*

Avantajları

- Genelde çok hızlı
- Yerinde (in-place) çalışır
- Cache-friendly
- Kolay paralelleştirme

Dezavantajları

- Stabil değil
- Kötü pivot *O(n^2)*
- Rekürsyon derinliği

Pivot Seçim Stratejileri

Rastgele: En güvenli, *O(n^2)* olasılığını minimize eder.

Median-of-Three: İlk, orta ve son elemanın medyanı. Pratik ve etkili.

İlk/Son Eleman: Basit ama sıralı/ters sıralı dizilerde tehlikeli.

Stabilite ve "Yerinde" (In-Place) Kavramı

Sıralama algoritmalarını değerlendirirken sadece hız yeterli değildir. İki önemli özellik daha vardır: Stabilite ve Yerinde Çalışma.

Stabilite (Stability)

Eşit anahtarlı elemanların **göreli sırası korunur mu?**

Örnek Senaryo

Öğrenci listesi önce puana, sonra isme göre sıralı. Sadece puana göre *stabil* sıralarsanız, aynı puandaki öğrencilerin isim sırası bozulmaz.

Stabil Algoritmalar

- Merge Sort ✓
- Insertion Sort ✓
- Bubble Sort ✓

Stabil Olmayan

- Quick Sort ×
- Heap Sort ×
- Selection Sort ×

Yerinde (In-Place)

Ekstra bellek (*O(n)* gibi) kullanmayan algoritma.

Yerinde Algoritmalar

- Quick Sort (genelde)
- Insertion Sort
- Selection Sort
- Heap Sort

Yerinde Olmayan

- Merge Sort (*O(n)* ekstra bellek)
- Counting Sort

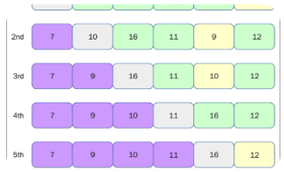
Not: Rekürsyon için *O(log n)* bellek (call stack) genelde "yerinde" sayılır.

> **Trade-off: Stabilite vs. Hız**
>
> Quick Sort hızlı ama stabil değil. Merge Sort stabil ama ekstra bellek kullanır. İhtiyacınıza göre seçim yapın!

Algoritma Seçimi: Pratik Rehber

Gerçek Dünyada Ne Kullanılır?

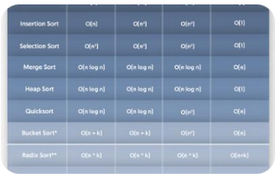
Teoride öğrendiğimiz algoritmaların pratikte nasıl kullanıldığını anlamak, bu bilgiyi değerli kılar.



Küçük Veri (n ≈ 10-20)

Seçim: Insertion Sort

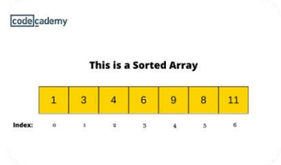
Basit, hızlı ve yeterince verimli. Karmaşık algoritmaların overhead'i gereksizdir.



Genel Amaçlı

Seçim: Quick Sort veya Merge Sort

Quick Sort iyi pivot seçimi ile genelde daha hızlı. Merge Sort güvenilir ve stabil.



Neredeyse Sıralı Veri

Seçim: Insertion Sort veya Timsort

Insertion Sort adaptive özelliği sayesinde *O(n)*'e yaklaşır.

Gerçek Dünya Uygulamaları

Python: list.sort()

Timsort kullanır - Insertion Sort ve Merge Sort'un hibrit versiyonu. Stabil, *O(n log n)* garantili ve neredeyse sıralı veride çok hızlı.

Java: Arrays.sort()

İlkel tipler için **Dual-Pivot QuickSort**, nesneler için **Timsort** kullanır.

C++ STL: std::sort()

Introsort - Quick Sort'un gelişmiş versiyonu. Kötü durumda Heap Sort'a geçer.

JavaScript: Array.sort()

Motor bağımlı - genelde Timsort veya Merge Sort varyantları.

💡 Pratik Tavsiye

Çoğu durumda dilin yerleşik sıralama fonksiyonunu kullanın. Bunlar yıllarca optimize edilmiş, test edilmiş ve güvenilirdir. Kendi algoritmanızı sadece çok özel durumlarda yazın!